

Instructor Key: Week 1 Assessment Event

Pump Readiness Closeout

Key/rubric for the Week 1 student assessment package.

1 Artifact status and assessment design boundary

Field	Status
Student artifact	Week 1 Python-heavy assessment event; 50 points; student copy intentionally 5 pages.
Purpose	Close the Week 1 pump scenario while measuring week-level mastery of visible output/state, assignment/expression reasoning, type/cast/truthiness evidence, exact-boundary comparison, Boolean readiness, testing, debugging, and explanation.
Exam-style alignment	Uses original code-reading, output prediction, value/type, expression evaluation, boundary testing, constrained completion, and debugging item families appropriate to ENGR 102 exam/practice style.
Pedagogical guardrail	Final answers alone are insufficient; trace/test/debug/explanation evidence still matters. No functions, loops, lists, or <code>if/else</code> are required in this Week 1 event.
Scenario boundary	This is the closeout of the Week 1 pump-readiness scenario. Week 2 should start a new scenario family.
Source boundary	Original data, wording, and surface structure. Aggregate exam/practice item families informed scope only; no secure prompt wording, answer-key phrasing, distinctive source contract, or private student data is used.
Canvas status	Not Canvas-released. Student PDF still needs Lance staging plus Ally/Panorama review if selected for use.
Copyright	Copyright © 2026 Lance L.A. White. All rights reserved.

2 Point map and mastery bands

Points	Section	Primary evidence
8	A. Output and state evidence	predict; trace
8	B. Types, casts, and truthiness	recognize; debug
12	C. Boundary and Boolean trace	predict; explain
8	D. Test-case reasoning	test; explain
8	E. Completion and debugging	implement; debug
6	F. Scenario closeout explanation	transfer; explain
50	Total	Week 1 mastery evidence

45–50	Secure / transferable: student shows usable state, type, boundary, Boolean, test, debug, and closeout evidence.
40–44	Mostly secure: one limited gap; targeted feedback should be enough.
35–39	Developing: partial week-level evidence; route support by weakest tag/section.
30–34	Fragile / support needed: several evidence types unstable.
0–29	Not yet visible: diagnose setup, reading, tracing, type, boundary, or written-explanation issue before labeling mastery.

Override: do not treat a high score as fully secure if the trace/debug/test evidence is blank or inconsistent.

3 Reference code with expected state

```
1 target_lpm = 21.0
2 tolerance_text = "0.60"
3 low_limit_lpm = target_lpm - float(tolerance_text) # 20.4
4 high_limit_lpm = target_lpm + float(tolerance_text) # 21.6
5
6 flow_text = "21.60" # remains a string
7 flow_lpm = float(flow_text) # 21.6, float
8 pressure_psi = 43
9 pressure_limit_psi = 45
10 operator_initials = "" # blank string
11 approval_label = "reviewed"
12 sensor_ready = True
13 retry_count = 0
14
15 flow_low_ok = flow_lpm >= low_limit_lpm # True
16 flow_high_ok = flow_lpm <= high_limit_lpm # True, exact upper included
17 flow_ok = flow_low_ok and flow_high_ok # True
18 pressure_ok = pressure_psi <= pressure_limit_psi # True
19 approval_ok = approval_label == "reviewed" # True
20 setup_ok = sensor_ready and operator_initials != "" # False
21 retry_ok = retry_count == 0 # True
22 ready_to_close = flow_ok and pressure_ok and approval_ok and setup_ok and retry_ok # False
```

4 A. Output and state evidence key

[predict](#) [trace](#)

Pts	Prompt focus	Secure answer	Scoring / support note
2	Limits	low_limit_lpm = 20.4; high_limit_lpm = 21.6.	Partial if arithmetic is right but variable names reversed.
2	First two print lines	limits 20.4 21.6 flow 21.6	Exact formatting not primary, but values and print order should be clear.
2	Later displayed Boolean values	flow ok True setup ok False close False	If final close is true, check whether student used or logic or missed blank initials.
2	Line controlling setup	setup_ok = sensor_ready and operator_initials != "".	Must identify blank initials as the failed part.

5 B. Types, casts, and truthiness key

recognize debug

Pts	Prompt focus	Secure answer	Scoring / support note
2	flow_lpm value/type	21.6, type float.	Accept 21.60 only if student clearly knows numeric float value; Python display is 21.6.
2	flow_text after cast	Value remains "21.60"; type <code>str</code> . <code>float(flow_text)</code> returns a new float; it does not mutate the original string.	Common issue: believing cast overwrites source variable.
2	String concatenation trap	"21.600.60". Both operands are strings, so <code>+</code> concatenates instead of adding numerically.	If student answers 22.2, route to type/cast support.
2	Truthiness versus approval evidence	<code>bool(approval_label)</code> only checks non-empty string. Non-empty wrong labels such as "pending" or "rejected" would also be true. Correct check is exact comparison to "reviewed".	Strong answer distinguishes non-empty from semantically approved.

6 C. Boundary and Boolean trace key

predict explain

Pts	Name/expression	Value	Reason
2	flow_low_ok	True	21.6 $\geq$ 20.4.
2	flow_high_ok	True	21.6 $\leq$ 21.6; exact upper boundary included.
2	flow_ok	True	True and True.
2	pressure_ok	True	43 $\leq$ 45.
2	ready_to_close	False	All-required and; setup_ok is false.
2	Controlling condition	setup_ok; specifically blank operator_initials.	

Correct choice: D. The `or` expression is not equivalent to the intended all-required readiness rule. It can become true when only one piece of evidence is true, while readiness requires every evidence check to pass.

7 D. Test-case reasoning key

test explain

Pts	Prompt focus	Secure answer	Scoring / support note
2	Upper boundary	Flow value 21.6; expected <code>flow_high_ok = True</code> .	Catches mistaken strict <.
2	Just above boundary	Example 21.61; <code>flow_high_ok</code> should become false.	Accept any value above 21.6.
2	Setup should pass	Keep <code>sensor_ready = True</code> ; set <code>operator_initials</code> to a non-empty string such as "AM". Expected <code>setup_ok = True</code> .	Do not require changing flow/pressure.
2	Catch mistaken <code>or</code> in readiness	Use a case where one required condition fails but others pass, e.g., the current case with <code>setup_ok = False</code> . Correct readiness false; mistaken <code>or</code> can return true.	Strong answer states expected contrast, not just input values.

8 E. Completion and debugging key

implement debug

```
ready_to_close = flow_ok and pressure_ok and approval_ok and setup_ok and retry_ok
```

Pts	Prompt focus	Secure answer	Scoring / support note
3	Completed expression	Requires all five Boolean variables with <code>and</code> . Equivalent parenthesized forms acceptable.	Deduct for <code>or</code> , omitted setup, or using raw strings as evidence.
2	Unsafe <code>flow_ok</code> or <code>approval_ok</code>	It allows closeout if either flow or approval passes and ignores pressure, setup, and retry evidence. In current data it would hide the missing-operator-initials problem.	Must critique connector and missing conditions.
3	Bad <code>bool(approval_label)</code> example	Any non-empty wrong label: "pending", "rejected", "hold", "not reviewed".	Award full credit when example is non-empty and semantically not reviewed.

The record is not ready to close. The flow evidence is in range because `flow_low_ok` and `flow_high_ok` are both true, including the exact upper boundary `21.6 <= 21.6`. Pressure and approval also pass, and `retry_ok` is true. The failed required condition is `setup_ok`, because `operator_initials` is blank. The next action is to collect or enter the operator initials, then recompute the readiness evidence before closing the pump record.

Pts	Scoring for closeout note
4	Names final decision, cites at least two correct Python evidence points, and identifies the next action. Full credit requires not-ready-to-close because setup evidence fails.
2	Support route: student identifies a plausible weak evidence type and a next small action. Treat this as evidence interpretation, not self-esteem.

10 Common errors and support routing

Observed evidence	Likely issue	Recovery/support action
Final readiness true despite blank initials	and/or confusion or missed controlling false condition	Require a condition ledger with each Boolean value before final expression.
<code>flow_text</code> treated as a float after casting	Non-mutating cast misunderstanding	Rebuild Day 3 cast trace: source string, returned float, stored variable.
<code>21.6 <= 21.6</code> marked false	Inclusive boundary confusion	Use exact-boundary mini-row with <code><=</code> and contrast with <code><</code> .
Approval accepted by <code>bool(...)</code>	Truthiness/semantic label confusion	Compare "pending", "rejected", and "reviewed" with explicit <code>==</code> .
Correct final values but no evidence	Guessing or mental shortcut	Ask for the shortest trace: limits, flow checks, setup check, final closeout.
Weak closeout note	Difficulty translating code evidence into engineering decision	Use sentence frame: "The record is/is not ready because ...; before closing, we need ..."

11 Recovery mini-surface

```
a = True
b = False
a and b # False: all-required rule fails when one part fails
a or b  # True: at-least-one rule passes when one part passes
label = "pending"
bool(label) # True, because the string is non-empty
label == "reviewed" # False, because the label is not reviewed
```

12 Assessment-quality notes for instructor review

Quality dimension	How this event addresses it
Exam/practice alignment	Uses code reading, exact output, current-state values, expression evaluation, Boolean selection, boundary tests, constrained completion, and debugging.
Good pedagogy retained	Requires trace, test, debug, and written engineering decision evidence rather than only multiple-choice final answers.
Challenge level	Combines all Week 1 traps: string numeric values, non-mutating casts, concatenation, explicit label checks, exact upper boundary, all-required Boolean readiness, and controlling false condition.
Scenario closeout	The pump record cannot close until setup evidence is repaired; this gives the Week 1 story a concrete final action.
Out-of-scope protection	Does not assess loops, lists, functions, or <code>if/else</code> ; those belong later.
Math approval boundary	No math-heavy item is included; math-faculty approval is not needed for this Python-heavy event.

13 Release/accessibility notes

Local verification should check tagged PDF output, extractable text, odd page count, print-safe margins, render sheets, answer-key separation, no private/protected markers, and the student-facing no-internal-surface rule. If selected for Canvas use, Lance uploads/stages the student PDF and retrieves both Ally and Panorama scores. Working release threshold remains Ally at least 95% and Panorama at least 95%.

14 Copyright

Copyright © 2026 Lance L.A. White. All rights reserved.